ACID Properties
PostgreSQL vs ClickHouse

Implementation comparison and experimental results

Executive summary

PostgreSQL and ClickHouse both provide meaningful ACID guarantees, but at different scopes. PostgreSQL is designed around general-purpose, multi-statement transactions and database-enforced constraints. The tested ClickHouse 24.8 configuration is designed around analytical ingestion and query processing: a single synchronous INSERT block is atomic and durable, while separate statements are not automatically combined into one rollback unit. The results matched these expected implementation differences.

PostgreSQL	OLTP-oriented engine with explicit transaction blocks, MVCC snapshots, schema constraints, and Write-Ahead Logging.
ClickHouse	OLAP-oriented columnar engine using MergeTree data parts, sparse indexing, and query-level snapshots.
Experimental outcome	All observations matched the expected behavior. The tested requirement was protected in 7 cases and was not protected in the 3 ClickHouse scenarios designed to expose differences in transaction scope and constraint enforcement.

What ACID means

ACID describes reliability properties of database operations. A transaction is one logical unit of work, which may contain one statement or several related statements.

Atomicity	All changes in the tested unit succeed together, or none of them remain.
Consistency	Defined data rules remain valid before and after the operation.
Isolation	Concurrent work does not create an invalid or unstable view for the tested operation.
Durability	After success is acknowledged, the data survives a failure and restart.

Project connection

The existing analytical benchmark measures query performance. This ACID benchmark intentionally does not measure timing: it tests correctness, visibility, rollback scope, and survival after a crash. The two benchmarks answer different questions and should remain separate.

Test environment and scope

Databases	PostgreSQL 16 and ClickHouse 24.8.
Infrastructure	Python tests ran on VM1. Both databases ran in their existing Docker containers on VM2 and used the same hardware.
ClickHouse configuration	Single-node MergeTree tables.
Test data	Synthetic rows in isolated tables named acid_reports and acid_audit. The analytical noisy_neighbors table was not modified.
Result set	10 result rows: 4 Atomicity, 2 Consistency, 2 Isolation, and 2 Durability.

How the tests were designed

- The tests intentionally create invalid values, duplicate identifiers, failed statements, and crash markers.
- Using separate tables prevents those operations from changing the analytical dataset or its benchmark results.
- Fixed test identifiers make every test repeatable and allow the scripts to clean only their own rows.

How to read the result labels

"Expected behavior observed = true" means the database behaved exactly as predicted. "ACID requirement protected" is a separate question: it indicates whether the selected business or transactional rule was protected. Therefore, a ClickHouse limitation can be an expected and successful experiment while the tested requirement is marked as not protected.

Atomicity

Goal: determine whether partial changes remain after an error. Two scopes were tested because PostgreSQL and ClickHouse define the natural atomic unit differently.

Test A1 - Single-statement atomicity

One multi-row INSERT contained valid rows and one invalid row. The requirement was: the statement must not save only the valid subset.

```
INSERT several rows in one statement
  - valid row
  - valid row
  - invalid duration value
Verify stored rows for this test case
```

Database	Observed result	Requirement
PostgreSQL	CHECK violation; stored_rows = 0	**Protected**
ClickHouse	Type parse error; stored_rows = 0	**Protected**

Why PostgreSQL produced this result: the statement ran inside an implicit transaction. The CHECK constraint rejected the invalid row, so the whole statement was rolled back.

Why ClickHouse produced this result: the rows were sent as one INSERT block. Parsing failed before a valid MergeTree part was committed, so no partial subset became visible. This demonstrates that ClickHouse does provide atomicity at the single-insert-block scope.

Test A2 - Multi-statement atomicity

The logical operation contained two statements: insert a report, then insert an audit record with an invalid event code. The requirement was: if step 2 fails, step 1 must also disappear.

```
Step 1: INSERT valid report
Step 2: INSERT invalid audit row
Verify whether the report from Step 1 remains
```

Database	Observed result	Requirement
PostgreSQL	Second statement failed; stored_report_rows = 0	**Protected**
ClickHouse	Second statement failed; stored_report_rows = 1	**Not protected**

Why PostgreSQL succeeded: both statements were inside one explicit transaction. After the second statement failed, ROLLBACK removed the earlier report insert. PostgreSQL therefore protected the complete business operation, not only each statement.

Why ClickHouse did not protect this requirement: in the tested configuration, the two INSERTs were independent operations. The first INSERT had already created and committed its own data part before the second INSERT failed. Because experimental multi-statement transactions were not enabled, there was no shared rollback boundary.

Atomicity conclusion

PostgreSQL provides general multi-statement atomicity. ClickHouse provided atomicity for the tested single INSERT block, but not a default rollback across two independent statements. The result is a difference in transaction scope, not an absence of all atomicity in ClickHouse.

Consistency

Consistency is meaningful only after a rule is defined. The selected application invariant was: every report_id must uniquely identify one complaint.

Test C1 - Duplicate report identifier

The same report_id was inserted twice with different duration values. The test checked whether the database itself rejected the duplicate.

Database	Observed result	Requirement
PostgreSQL	UniqueViolation; rows_for_report_id = 1	**Protected**
ClickHouse	Both INSERTs succeeded; rows_for_report_id = 2	**Not protected**

Why PostgreSQL protected the rule: report_id was defined as PRIMARY KEY. PostgreSQL implements a primary key as a unique, non-null constraint backed by a unique index, so the second row was rejected immediately.

Why ClickHouse accepted both rows: the MergeTree ORDER BY key controls physical sort order and supports a sparse index for fast filtering. It is not a uniqueness constraint. Rows with the same key can therefore coexist unless deduplication is handled through a different engine, ingestion logic, or query design.

Consistency conclusion

The test does not mean that ClickHouse became internally inconsistent. It means that this selected application-level invariant was enforced by PostgreSQL but was not enforced by the tested MergeTree schema. In ClickHouse, this responsibility normally moves to the ingestion pipeline or application.

Isolation

Goal: determine whether two reads in one logical operation use the same view while another session commits a new row.

Test I1 - Stable snapshot across two reads

Session A counted the test rows. Session B inserted and committed one row. Session A then repeated the count. PostgreSQL used REPEATABLE READ; ClickHouse executed two independent SELECT statements.

Database	Observed values	Requirement
PostgreSQL	first = 1; second = 1; after commit = 2	**Protected**
ClickHouse	first = 1; second = 2	**Not protected**

Why PostgreSQL kept the same count: REPEATABLE READ uses one transaction-level MVCC snapshot. Session B committed successfully, but its row was not added to Session A's already-established snapshot. A new query after Session A committed saw the row.

Why ClickHouse returned a different second count: each SELECT was a separate statement and obtained a new query snapshot. The synchronous INSERT created a committed part between the two reads, so the second query included it. The test does not show partial-row visibility; each SELECT still read a complete query-level state.

Isolation conclusion

PostgreSQL protected a stable transaction-level view because both reads shared one REPEATABLE READ snapshot. ClickHouse protected each individual query snapshot, but the two independent SELECT statements did not share one transaction snapshot, so the second read included the newly committed row.

Durability

Goal: verify that a row acknowledged as successfully inserted still exists after an abrupt database-process crash and restart.

Test D1 - Acknowledged marker survives restart

For each database, the prepare phase inserted a controlled marker and confirmed one stored row. The corresponding Docker container on VM2 was then terminated with SIGKILL and started again. The verify phase checked for the exact marker.

```
Prepare: insert marker and wait for success acknowledgement
VM2: docker kill -s KILL <database container>
VM2: docker start <database container>
Verify: query the exact marker after restart
```

Database	Observed result	Requirement
PostgreSQL	acknowledged = true; rows after restart = 1	**Protected**
ClickHouse	acknowledged = true; rows after restart = 1	**Protected**

Why PostgreSQL survived: PostgreSQL records changes in the Write-Ahead Log before relying on changed table pages. After a crash, recovery can replay WAL records and restore committed work. The acknowledged marker was therefore present after restart.

Why ClickHouse survived: the test used a normal synchronous INSERT into a MergeTree table. The successful insert produced a persistent data part before acknowledgement. After the server restarted, ClickHouse loaded the stored part and the marker remained available.

Durability test scope

The experiment tested one concrete failure scenario: a synchronous insert was acknowledged, the database container was terminated abruptly with SIGKILL, the same container was restarted, and the exact marker row was queried again. Both databases retained the acknowledged marker after the restart.

Complete experimental summary

Property / test	PostgreSQL	ClickHouse
Atomicity - one INSERT block	Protected: 0 rows remained	Protected: 0 rows remained
Atomicity - two statements	Protected: rollback left 0 rows	Not protected: first row remained
Consistency - unique report_id	Protected: 1 row remained	Not protected: 2 rows remained
Isolation - shared snapshot	Protected: 1 -> 1, then 2	Not protected: 1 -> 2
Durability - process restart	Protected: marker survived	Protected: marker survived

Overall conclusion

PostgreSQL offers broad, default transactional guarantees across multiple statements, enforces relational invariants in the schema, and can preserve one snapshot across a transaction. In the tested configuration, ClickHouse offers strong guarantees for the analytical operations it is designed around, including an atomic synchronous INSERT block, a consistent snapshot for each individual query, and durable stored parts.

What each database is optimized to do

Area	PostgreSQL	ClickHouse
Primary workload	Transactional applications and frequent related writes	Large analytical scans, aggregation, logs, and events
Natural unit of correctness	Transaction containing one or many statements	Individual insert block and individual query in the tested configuration
Integrity enforcement	Primary keys, foreign keys, CHECK, UNIQUE, NOT NULL	Types and schema; many cross-row rules handled during ingestion or separately
Concurrency model	MVCC and configurable SQL isolation levels	Query-level snapshots over immutable data parts
Persistence mechanism	WAL plus crash recovery	Persistent MergeTree data parts

Connection to the analytical benchmark

The analytical benchmark showed ClickHouse outperforming PostgreSQL on the three read-heavy queries. The ACID experiment explains the architectural trade-off behind that result: ClickHouse is optimized around columnar parts, sparse indexes, and analytical execution, whereas PostgreSQL carries broader transactional machinery and constraint enforcement. These are complementary findings, not competing measurements.

Sources and project evidence

1. PostgreSQL 16 - BEGIN and transaction blocks
2. PostgreSQL 16 - Constraints
3. PostgreSQL 16 - Transaction isolation
4. PostgreSQL 16 - Write-Ahead Logging and reliability
5. ClickHouse - Transactional (ACID) support
6. ClickHouse - MergeTree table engine
7. Project results - ACID_Benchmark/results/acid_results.csv